

Bankier Dagobert

1 Lehrziele

- While-Anweisung
- If-Anweisung

2 Problemstellung

Der Bankier Dagobert weiß, dass sein Vermögen mindestens eine Million aber höchstens zwei Millionen Dukaten ist. Um sich die Zeit zu vertreiben, legt er seine Dukaten einmal in Quadrat- und einmal in Dreiecksform auf, wobei er jeweils alle Münzen verwendet, ohne eine übrig zu lassen. Und siehe da, es geht sich genau aus. Wie viele Dukaten besitzt Dagobert? Gibt es dazu eine eindeutige Lösung?

3 Hinweise

Es gibt (wie immer) mehrere Möglichkeiten, dieses Problem zu lösen. Es werden hier zwei verschiedene Varianten, eine eher einfache, die aber mehr Rechenzeit verbraucht und eine zweite, die in der Analyse mehr mathematische Vorarbeit benötigt, dafür der Algorithmus dann einfacher und effizienter ist, vorgestellt.

Da Dagobert seine Dukaten in Form eines Dreiecks auflegt, müssen wir auf jeden Fall eine effiziente Möglichkeit finden, sog. Dreieckszahlen zu ermitteln. Eine Dreieckszahl $D(n)$ ist die Reihe $1 + 2 + 3 + \dots + n$. So können wir herausfinden, wieviele Dukaten ein Dreieck dessen Seiten jeweils n Dukaten haben, beinhaltet (siehe auch Abbildung 1).

Carl Friedrich Gauß fand (im Volksschulalter!) heraus, dass wir für eine Reihe $1 + 2 + 3 + \dots + n$ das Ergebnis mittels der Formel

$$\frac{n}{2} \cdot (n + 1)$$

berechnen können.

Wenn Dagobert die Dukaten in quadratischer Form auflegt, wird die Sache sehr viel einfacher. Wenn er ein Quadrat mit der Seitenlänge n auflegt, wissen wir, dass er n^2 Dukaten in diesem Quadrat liegen hat. Wir nennen die Anzahl der Dukaten in einem Quadrat mit der Seitenlänge n die Quadratzahl von n , kurz $Q(n)$.



Abbildung 1: Dreieckszahl

3.1 Brute Force Algorithm

Der einfache Algorithmus geht von der Überlegung aus, dass für alle $n > 1$ die Aussage $D(n) < Q(n)$ gilt. Weiters wissen wir, dass das kleinste Quadrat, das Dagobert bei einem Vermögen von mindestens 1.000.000 Dukaten legen kann, ein Quadrat mit der Seitenlänge 1.000 ist.

Der Algorithmus ist nun folgendermaßen: Wir initialisieren die beiden Variablen $d = 1000$ und $q = 1000$ und berechnen $D(d)$ und $Q(q)$. Ist $D(d) = Q(q)$, dann haben wir eine Lösung gefunden, geben diese aus und erhöhen q um 1. Falls $D(d) < Q(q)$, erhöhen wir d um 1 und berechnen wieder $D(d)$ und $Q(q)$. Falls $D(d) > Q(q)$, erhöhen wir q um 1 und berechnen wieder $D(d)$ und $Q(q)$. Das machen wir solange, bis entweder $D(d)$ oder $Q(q)$ den Wert 2.000.000 überschreitet.

3.2 More Sophisticated

Wenn wir den Brute-Force Algorithmus genauer ansehen, können wir erkennen, dass der Startpunkt des Algorithmus mit 1.000 nicht optimal ist, da ja $D(1000)$ sicherlich unter 1.000.000 liegen muss (wegen $D(n) < Q(n)$ für alle $n > 1$). Der optimale Startpunkt würde ja bei einem $\bar{n}$ liegen, für welches $D(\bar{n}) = 1.000.000$ gilt. Um dieses $\bar{n}$ herauszufinden müssen wir die Gleichung

$$\frac{\bar{n}}{2} \cdot (\bar{n} + 1) = 1.000.000$$

lösen.

Durch ein wenig Umformen ergibt sich, dass wir die Gleichung

$$\bar{n}^2 + \bar{n} - 2.000.000 = 0$$

lösen müssen. Weil wir bequem sind, setzen wir dies in die Lösungsformel für quadratische Gleichungen ein und erhalten die beiden (gerundeten) Lösungen $\bar{n}_1 = 1413.71$ und $\bar{n}_2 = -1414.71$. Das negative Ergebnis werden wir gleich ignorieren (die Eigenschaft, dass das Quadrat negativer Zahlen positiv ist, trägt zu unserer Lösung keinen praktischen Wert bei).

Da wir ja an Vermögen von mindesten 1.000.000 Dukaten interessiert sind, können wir den Startpunkt unserer Überlegungen auf 1414 Dukaten festlegen. Durch analoge Überlegungen können Sie den Endpunkt der Schleife ermitteln.

Nun, wo wir Start- und Endpunkt der Schleife kennen, können wir den Algorithmus sehr einfach skizzieren. Wir initialisieren eine Variable n mit dem Wert 1414. Falls $\sqrt{D(n)}$ eine ganzzahlige Lösung ergibt, geben wir sie aus und erhöhen n um 1. Wenn dies nicht der Fall ist, erhöhen wir n um 1. In jedem Fall prüfen wieder, ob $\sqrt{D(n)}$ eine ganzzahlige Lösung ergibt. Dies machen wir solange, bis n den oben ermittelten Endwert erreicht.

3.3 Tricks

Beim zweiten Algorithmus benötigen wir die Wurzelfunktion. In C# kann die Wurzel einer Zahl mit Hilfe der Methode `Math.Sqrt` ermittelt werden. Das Ergebnis von `Math.Sqrt` ist, da das Ergebnis nicht ganzzahlig sein muss, natürlich eine Zahl vom Typ `double`.

Weiters müssen wir im zweiten Algorithmus herausfinden, ob das Ergebnis der Wurzel ganzzahlig ist. Dies können wir, indem wir den Modulo-Operator ein wenig ungewöhnlich ausnutzen. Nehmen wir an, dass die Variable `q` vom Typ `double` auf Ganzzahligkeit überprüft werden soll. Wenn `q % 1 == 0` ist, ist `q` ganzzahlig, sonst nicht.

4 Aufgaben und Abgaben

- Skizzieren Sie den Brute-Force Algorithmus mittels eines Ablaufdiagramms (Flow Chart).
- Implementieren Sie den Brute-Force Algorithmus. Geben Sie nicht nur das Ergebnis (oder die Ergebnisse, falls die Lösung nicht eindeutig ist) aus, sondern auch die Anzahl der Iterationen, die Sie benötigen, um das Ergebnis (die Ergebnisse) zu ermitteln.
- Implementieren Sie den ausgefeilteren Algorithmus und geben Sie auch hier neben dem/den Ergebnis/Ergebnissen die Anzahl der Iterationen zur Ermittlung der-/desselbigen an.